

Power Query EX 1

Power BI Power Query & Advanced Editor

Course: Data Analytics	Topic: Power Query & Advanced Editor	Estimated Time: 60–90 minutes	Level: Beginner–Intermediate
----------------------------------	--	---	--

Overview

In this hands-on practice exercise, you will use Power BI Desktop’s Power Query Editor to load, clean, and transform a real-world sales dataset. You will also explore the Advanced Editor to read, understand, and modify M (Mashup) language code — the language that runs behind every transformation step you apply in Power Query.

Learning Objectives

By the end of this exercise, you will be able to:

- Import a CSV file into Power BI using Power Query
- Identify and resolve common data quality issues (duplicates, inconsistent formats, extra whitespace)
- Apply column transformations: data types, text case, calculated columns
- Use the Power Query Advanced Editor to read and modify M code
- Write a custom M expression using Table.AddColumn and Date.Year
- Load the cleaned dataset into the Power BI data model

Dataset Description

The dataset file SalesData_ClassPractice1.csv contains 51 rows (including a header row) of simulated sales transaction records from January – May 2024.

Column	Data Type	Description / Known Issues
OrderID	Whole Number	Unique order identifier — contains one duplicate row
OrderDate	Date (messy)	Mixed formats: YYYY-MM-DD, MM/DD/YYYY, and 'Month DD YYYY'
CustomerName	Text	Some names have leading/trailing spaces; inconsistent casing
Product	Text	Product name — clean
Category	Text	Product category: Electronics, Accessories, Storage
Region	Text	Sales region: North, South, East, West
Quantity	Whole Number	Units sold — clean

UnitPrice	Decimal	Price per unit — clean
Discount	Decimal	Discount rate (0 to 0.15) — clean
SalesRep	Text	Sales representative name — inconsistent casing

Exercise Instructions

Task 1 — Load the Dataset into Power Query

1. Open Power BI Desktop.
2. On the Home ribbon, click Get Data → Text/CSV.
3. Browse to the file SalesData_ClassPractice1.csv and click Open.
4. In the preview dialog, click Transform Data (not Load) to open the Power Query Editor.

You should see 51 rows and 10 columns in the Applied Steps pane on the right. The steps 'Source' and 'Promoted Headers' should appear automatically.

Task 2 — Inspect and Profile the Data

1. On the View ribbon, enable Column Distribution, Column Quality, and Column Profile.
2. At the bottom of the editor, ensure "Column profiling based on entire dataset" is selected (not just the top 1,000 rows).
3. Click each column header and review the profiling statistics in the bottom pane.

Answer these questions before proceeding (write your answers in the Reflections section):

- How many errors does the OrderDate column show?
- Which column(s) show duplicate values in the top rows of the preview?
- What percentage of rows are valid in the OrderID column?

Task 3 — Remove Duplicate Rows

1. Click the OrderID column header to select it.
2. On the Home ribbon, click Remove Rows → Remove Duplicates.

Row count should drop from 51 to 50 (one duplicate row for OrderID 1001 is removed).

Task 4 — Fix the OrderDate Column

1. Right-click the OrderDate column header.
2. Select Change Type → Using Locale...
3. Set Data Type to Date and Locale to English (United States), then click OK.

Power Query's locale-aware parser can resolve mixed date formats that include written month names (e.g., 'January 18 2024'). Always set the locale to match the data's origin.

Task 5 — Clean the CustomerName and SalesRep Columns

1. Select the CustomerName column. On the Transform ribbon, click Format → Trim to remove leading and trailing spaces.
2. With CustomerName still selected, click Format → Capitalize Each Word to standardize casing.
3. Repeat steps 1–2 above for the SalesRep column.

Task 6 — Add a Calculated Revenue Column

1. On the Add Column ribbon, click Custom Column.
2. Name the new column Revenue.
3. In the Custom Column Formula box, enter the following expression:

```
[Quantity] * [UnitPrice] * (1 - [Discount])
```

4. Click OK. Change the new column's data type to Decimal Number.

Task 7 — Add a Year Column Using the Advanced Editor

1. On the Home ribbon, click Advanced Editor to open the M code view of your query.
2. Review all steps that were applied. Notice that each step is assigned to a variable using the **let...in** structure.
3. Locate the last variable name before the final **in** statement.
4. After that last step, add a comma and then add the following new step:

```
#"Added Year" = Table.AddColumn(  
    #"Changed Type1",  
    "OrderYear",  
    each Date.Year([OrderDate]),  
    Int64.Type  
)
```

Note: Replace #"Changed Type1" with the actual name of your last step if it differs.

5. Update the final **in** statement to reference **#"Added Year"** as the query's output.
6. Click Done to close the Advanced Editor.

Task 8 — Rename the Query and Load the Data

1. In the Queries pane on the left, right-click the query name and rename it to SalesTransactions.
2. On the Home ribbon, click Close & Apply to load the cleaned data into the Power BI data model.

Your final table should have 50 rows and 11 columns (the original 10 plus the new Revenue and OrderYear columns). Verify this in the Data view in Power BI.

Reference

Power Query Interface Quick Reference

Area	Purpose
Applied Steps Pane	Lists every transformation step applied to the query. Click any step to preview the data at that point.
Formula Bar	Shows the M code for the currently selected step. Can be edited directly for small changes.
Advanced Editor	Full M code view of the entire query. Use for complex changes or to add new steps manually.
Column Distribution	Shows a bar chart of value distribution for the selected column.
Column Quality	Reports the percentage of Valid, Error, and Empty values per column.
Column Profile	Detailed statistics: min, max, average, count of distinct values for the selected column.
Query Settings Pane	On the right side; shows the query name and the Applied Steps list.

M Language Quick Reference

M (Mashup Language) is a functional language used by Power Query. Each query is structured as:

```
let
    Step1 = <expression>,
    Step2 = <expression using Step1>,
    ...
in
    LastStep
```

M Function	Description
<code>Table.RemoveDuplicates (table, columns)</code>	Removes rows with duplicate values in the specified columns.
<code>Table.TransformColumnTypes (table, list)</code>	Changes data types for one or more columns.
<code>Text.Trim (text)</code>	Removes leading and trailing whitespace from a text value.
<code>Text.Proper (text)</code>	Capitalizes the first letter of each word.
<code>Text.Upper (text)</code>	Converts text to all uppercase.
<code>Text.Lower (text)</code>	Converts text to all lowercase.
<code>Table.AddColumn (table, name, expression, type)</code>	Adds a new column computed by the given expression.
<code>Date.Year (date)</code>	Returns the year component of a date value as a number.
<code>Date.Month (date)</code>	Returns the month component of a date value as a number.
<code>each [ColumnName]</code>	Shorthand for a row-level function: <code>_ => _[ColumnName]</code> .

Recommended Resources

- Microsoft Learn — Power Query M Language Reference:
<https://learn.microsoft.com/en-us/powerquery-m/>
- Power BI Documentation — Power Query Editor:
<https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-query-overview>
- Gil Raviv's 'Power Query Recipes' blog:
<https://datachant.com>

Reflections

Answer the following questions in your own words after completing the exercise. These are not graded for a single correct answer — they are designed to deepen your understanding of the concepts.

Section A — Data Quality Observations

1. What data quality issues did you identify during the profiling step (Task 2)?

List all issues you noticed. For each issue, describe which column was affected and why it could be a problem for analysis.

2. Why is removing duplicates an important step before analysis?

Think about how a duplicated order row could affect a Sales Total or a Row Count measure in Power BI.

Section B — Power Query Transformations

3. What is the difference between Change Type and Change Type Using Locale?

Why was the 'Using Locale' option necessary for the OrderDate column in this exercise?

4. What would happen if you skipped the Trim step on CustomerName?

Consider a scenario where a report groups sales by customer. How would extra spaces affect the grouping results?

Section C — Advanced Editor & M Language

5. What is the purpose of the let...in structure in M code?

Explain in your own words how each step builds on the previous one.

6. In Task 7, you used Date.Year() to extract the year. How would you modify the code to extract the month number instead?

Write the modified M step below.

7. Challenge: Why is it important to reference the correct previous step name in M code?

What would happen if you referenced a step name that does not exist, or referenced an earlier step rather than the most recent one?

Solution Guide

This section provides reference answers for each task and the reflection questions. Review only after completing the exercise on your own.

Task Solutions

Task 3 — Remove Duplicates (Expected M Step)

```
#"Removed Duplicates" = Table.Distinct(#"Changed Type", {"OrderID"})
```

Task 4 — Fix OrderDate (Expected M Step)

```
#"Changed Type with Locale" = Table.TransformColumnTypes(  
    #"Removed Duplicates",  
    {{ "OrderDate", type date }},  
    "en-US"  
)
```

Task 5 — Trim and Proper Case (Expected M Steps)

```
#"Trimmed Text" = Table.TransformColumns(  
    #"Changed Type with Locale",  
    {{ "CustomerName", Text.Trim, type text },  
     { "SalesRep", Text.Trim, type text }}  
)  
#"Capitalized Each Word" = Table.TransformColumns(  
    #"Trimmed Text",  
    {{ "CustomerName", Text.Proper, type text },  
     { "SalesRep", Text.Proper, type text }}  
)
```

Task 6 — Revenue Column (Expected M Step)

```
#"Added Custom" = Table.AddColumn(  
    #"Capitalized Each Word",  
    "Revenue",  
    each [Quantity] * [UnitPrice] * (1 - [Discount]),  
    type number  
)
```

Task 7 — Full Advanced Editor Output (Reference)

Below is an example of the complete M query after all steps are applied. Your step names may differ slightly depending on the order you applied transformations.

```
let
```

```

Source = Csv.Document(
    File.Contents("C:\\path\\to\\SalesData_ClassPractice1.csv"),
    [Delimiter=",", Columns=10, Encoding=65001,
QuoteStyle=QuoteStyle.None]
),
#"Promoted Headers" = Table.PromoteHeaders(
    Source, [PromoteAllScalars=true]
),
#"Changed Type" = Table.TransformColumnTypes(
    #"Promoted Headers",
    {{"OrderID", Int64.Type}, {"Quantity", Int64.Type},
    {"UnitPrice", type number}, {"Discount", type number}}
),
#"Removed Duplicates" = Table.Distinct(
    #"Changed Type", {"OrderID"}
),
#"Changed Type with Locale" = Table.TransformColumnTypes(
    #"Removed Duplicates",
    {{"OrderDate", type date}}, "en-US"
),
#"Trimmed Text" = Table.TransformColumns(
    #"Changed Type with Locale",
    {{"CustomerName", Text.Trim, type text},
    {"SalesRep", Text.Trim, type text}}
),
#"Capitalized Each Word" = Table.TransformColumns(
    #"Trimmed Text",
    {{"CustomerName", Text.Proper, type text},
    {"SalesRep", Text.Proper, type text}}
),
#"Added Revenue" = Table.AddColumn(
    #"Capitalized Each Word",
    "Revenue",
    each [Quantity] * [UnitPrice] * (1 - [Discount]),
    type number
),
#"Added Year" = Table.AddColumn(
    #"Added Revenue",
    "OrderYear",
    each Date.Year([OrderDate]),
    Int64.Type
)
in
#"Added Year"

```

Reflection Answers

1. Data Quality Issues Found

- **OrderID:** One duplicate row (OrderID 1001 appears twice).
- **OrderDate:** Mixed date formats — YYYY-MM-DD, MM/DD/YYYY, and Month DD YYYY. Power Query cannot auto-detect these without a locale hint, so some rows show as errors.
- **CustomerName:** Leading/trailing whitespace on several rows (e.g., " TechZone Inc", " Nexus Corp"). Inconsistent casing (e.g., "nexus corp" vs "Nexus Corp").
- **SalesRep:** Inconsistent casing (e.g., "james smith" vs "James Smith").

2. Why Remove Duplicates

A duplicated order row inflates every aggregated measure. A SUM of Revenue would double-count that order; a COUNT of orders would be off by one. Duplicate rows are one of the most common and damaging data quality issues in analytics because they produce incorrect KPIs that often go unnoticed.

3. Change Type vs. Change Type Using Locale

Change Type applies the data type using the machine's default locale settings. Change Type Using Locale lets you specify the origin locale, which determines how ambiguous formats are parsed. In this dataset, formats like "January 18 2024" cannot be resolved without knowing the language, so the en-US locale is required.

4. Skipping Trim on CustomerName

Power BI would treat " TechZone Inc" and "TechZone Inc" as two different customers. Any visual, table, or measure that groups or filters by customer would split that customer's sales across two separate rows, producing incorrect totals and a misleading customer list.

5. The let...in Structure

The let block defines a sequence of named transformation steps. Each step is a variable that holds the result of an expression, and subsequent steps can reference earlier ones by name. The in keyword specifies which step's result is the final output of the query. This structure makes queries readable, debuggable (you can click any step in the Applied Steps pane), and reusable.

6. Extracting Month Instead of Year

```
#"Added Month" = Table.AddColumn(  
    #"Added Revenue",  
    "OrderMonth",  
    each Date.Month([OrderDate]),  
    Int64.Type  
)
```

7. Referencing Incorrect Step Names

M code is sequential: each step must reference an existing step name. If you reference a step that does not exist, Power Query throws a "Token Identifier Expected" or "Expression.Error" at that line. If you reference an earlier step instead of the most recent one, your new column will be added to the table as it existed at that earlier point, bypassing all transformations applied after it — you would silently lose work.

End of Class Practice 1